

Lab4 Report

Name: 时分

Student ID: PB23151799

Propose

Sakiko has savings, earns money and spends money each month. The recursive formula of the three variables are as follow.

$$\begin{aligned} S(n) &= \begin{cases} 10, n = 0 \\ S(n-1) + \text{Earn}(n-1) - \text{Spend}(n-1), n > 0 \end{cases} \\ \text{Earn}(n) &= \begin{cases} 6, n = 0 \\ \text{Earn}(n-1) \times 2, n > 0 \end{cases} \\ \text{Spend}(n) &= \begin{cases} 2, n = 0 \\ 2, n > 0 \text{ and } \text{Spend}(n-1) \geq \text{Earn}(n-1) \\ \text{Spend}(n-1) \times 4, n > 0 \text{ and } \text{Spend}(n-1) < \text{Earn}(n-1) \end{cases} \end{aligned}$$

The purpose is to use recursive method to calculate Sakiko's final savings after N months. The program is processed in LC-3 system, and it is designed with assembly codes.

$N(0 \leq N \leq 10)$ will be stored in x3100, and the result will be stored in x3200. R0-R7 are set to zeroes at beginning.

Principles

A stack is needed for the program. The recursive subroutine should be like:

```
FUNC
;Jugde whether to end the recursion
;Push R7,Earn(n),Spend(n)
;Calculate Earn(n+1),Spend(n+1)
JSR FUNC
;Pop R7,Earn(n),Spend(n)
;Calculate S
RET
```

To return correctly, R7 should be stored in the stack as well. R0 will be used as the counter to determine whether the recursion is over.

The calculation of $\text{Earn}(n+1)$ and $\text{Spend}(n+1)$ is as below:

```
ADD R5,R2,R3
BRNZ KEEP      ;if Earn(n) <= Spend(n) goto KEEP
ADD R5,R3,R3
ADD R3,R5,R5    ;Spend(n+1) = 4*Spend(n)
BR NEXT
KEEP AND R3,R3,#0
ADD R3,R3,#-2   ;Spend(n+1) = 2
NEXT ADD R2,R2,R2 ;Earn(n+1) = 2*Earn(n)
```

Procedure

1. At first, the judging part is like:

```
FUNC
    ADD R0,R0,#-1
    BRn DONE
    ;Push R7,Earn(n),Spend(n)
    ;Calculate Earn(n+1),Spend(n+1)
    JSR FUNC
    ;Pop R7,Earn(n),Spend(n)
    ;Calculate S
DONE    RET
```

The subroutine works correctly. However, the program does nothing between the last pair of JSR and RET. To avoid this problem, the judging part should be modified like:

```
FUNC
    ADD R0,R0,#-1
    BRZ DONE
    ;Push R7,Earn(n),Spend(n)
    ;Calculate Earn(n+1),Spend(n+1)
    JSR FUNC
    ;Pop R7,Earn(n),Spend(n)
DONE    ;Calculate S
        RET
```

2. To deal with the situation $N = 0$, a judging part should be added after loading N , like:

```
    ADD R1,R1,#10    ;S
    LDI R0,INPUT
    BRZ OVER
    ...
    JSR FUNC
OVER    STI R1,RESULT
        HALT
```

3. At first I store $Spend(n)$ as positive numbers. However, this will cause extra calculations when doing subtraction. If $Spend(n)$ is saved as negative numbers, the problem will be solved.

Result

1. x3100: 0
x3200: 10
2. x3100: 1
x3200: 14
3. x3100: 2
x3200: 18
4. x3100: 3
x3200: 10

- 5. x3100: 4
x3200: 56
- 6. x3100: 5
x3200: 144
- 7. x3100: 6
x3200: 304
- 8. x3100: 7
x3200: 560
- 9. x3100: 8
x3200: 816
- 10. x3100: 9
x3200: 304
- 11. x3100: 10
x3200: 3374

All results are correct.

Discussion Questions

1. The reasons are as below:
 - (1) A stack isn't needed in a simple iterative program. We can calculate $Earn(n), Spend(n)$, and then calculate $S(n)$ in each loop.
 - (2) Because there isn't any subroutine in a simple iterative program, R7 won't be used, and there is no need to store R7.
 - (3) If N is very large, the stack will occupy a large part of memory and even overflow. However, there is no such worry in a simple iterative program.
2. In the recursion, we can calculate $S(n)$ before calling the subroutine, so that we don't have to save $Earn(n)$ and $Spend(n)$ in the stack, which will be only used to save R7.